Mastering GitHub Copilot: Essential Tips, Tricks, and Prompt Engineering for Optimal Coding Efficiency

Arik Bidny · Follow
8 min read · Apr 30, 2024

68

I ❤ GITHUB Copilot

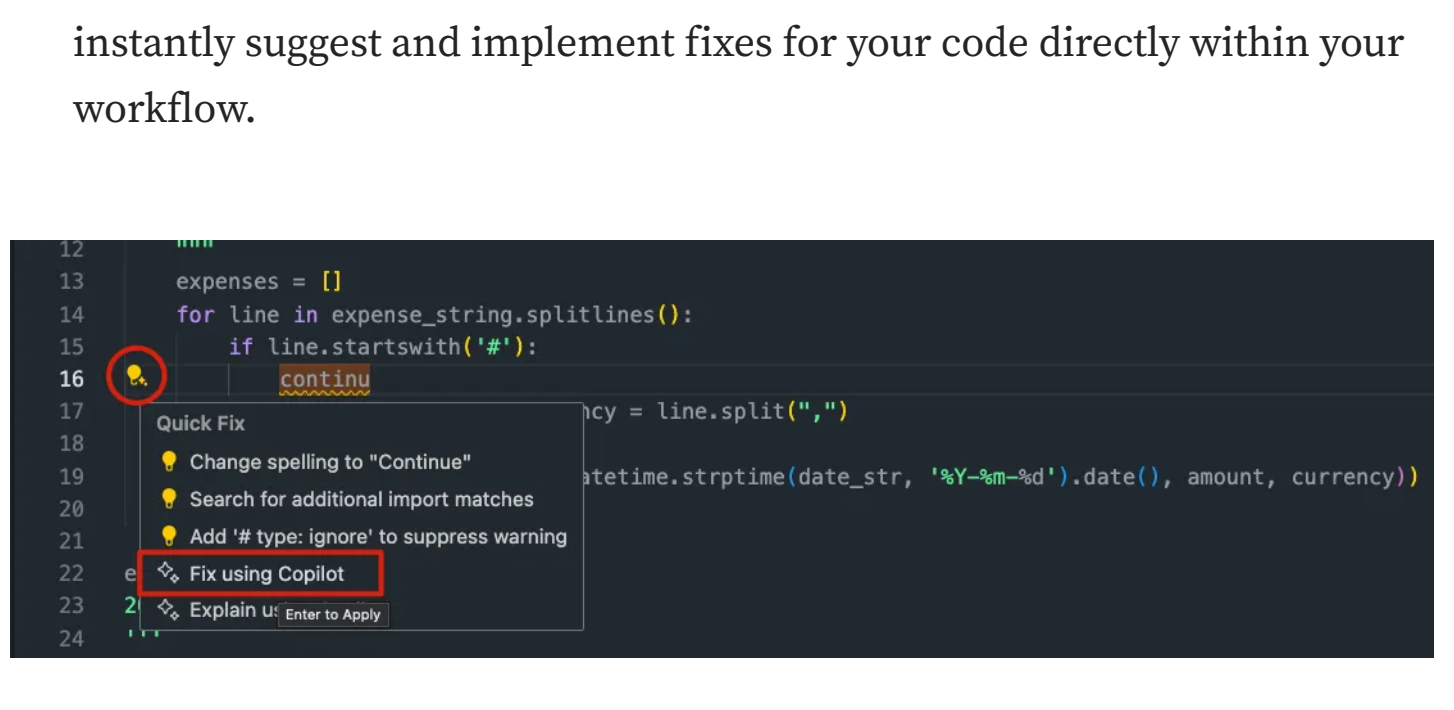
GitHub Copilot, introduced by GitHub as an AI-powered code completion tool, promises to revolutionize how developers write code. This innovative tool leverages the OpenAI GPT model to suggest entire lines or blocks of code as you type, functioning like a highly sophisticated pair-programmer. While its potential to increase productivity is immense, truly harnessing its capabilities requires a blend of savvy prompt engineering, adherence to best practices, and a good grasp of helpful tips and tricks.

In this post, we'll delve deep into the strategies that can maximize the effectiveness of GitHub Copilot. From understanding how to craft precise prompts that lead to better code suggestions, to implementing best practices that ensure code quality and security, our journey will cover all the essentials. We'll also explore why mastering these elements is crucial not just for enhancing your coding efficiency but also for leveraging AI as a powerful ally in your development projects.

Whether you're a seasoned developer or a newcomer to the world of coding, the insights shared here will equip you with the knowledge needed to turn GitHub Copilot from a mere coding assistant into a pivotal element of your development toolkit. By the end of this guide, you'll not only appreciate the capabilities of GitHub Copilot but also be proficient in extracting real value from it, ensuring that your coding process is both streamlined and sophisticated.

Tips — Code Styles

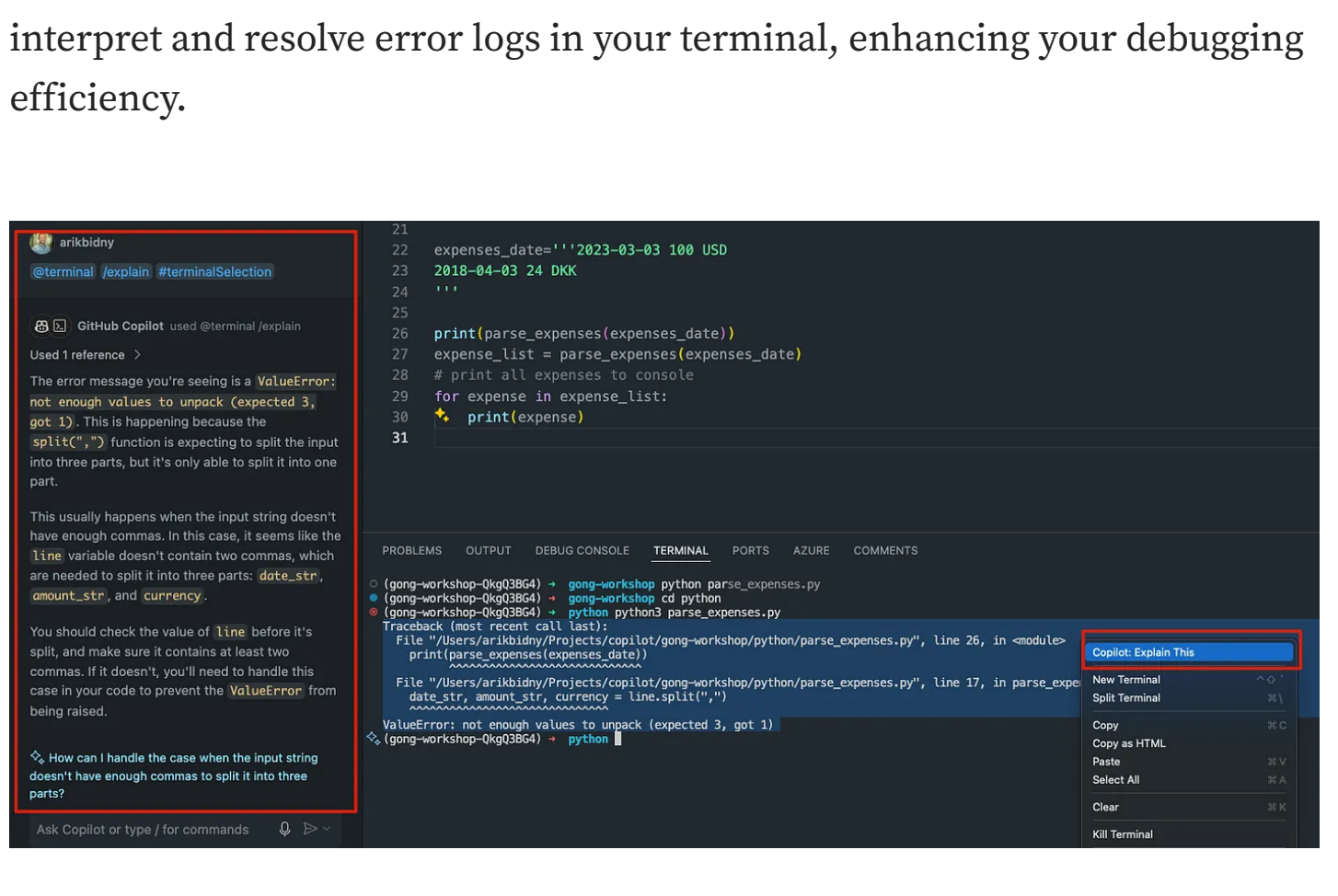
1. **Fix Code Inline** — Harness the power of GitHub Copilot in your IDE to instantly suggest and implement fixes for your code directly within your workflow.



```
    expenses = []
    for line in expense_string.splitlines():
        if line.startswith('#'):
            continu
```

Quick Fix
Change spelling to "Continue"
Search for additional import matches
Add '# type: ignore' to suppress warning
Fix using Copilot
Explain u
Enter to Apply

2. **Get help on error logs in your terminal** — Use GitHub Copilot to swiftly interpret and resolve error logs in your terminal, enhancing your debugging efficiency.



arikbidny
@terminal /explain #terminalSelection

GitHub Copilot used @terminal /explain
Used 1 reference

The error message you're seeing is a `ValueError: not enough values to unpack (expected 3, got 1)`. This is happening because the `split(",")` function is expecting to split the input into three parts, but it's only able to split it into one part.

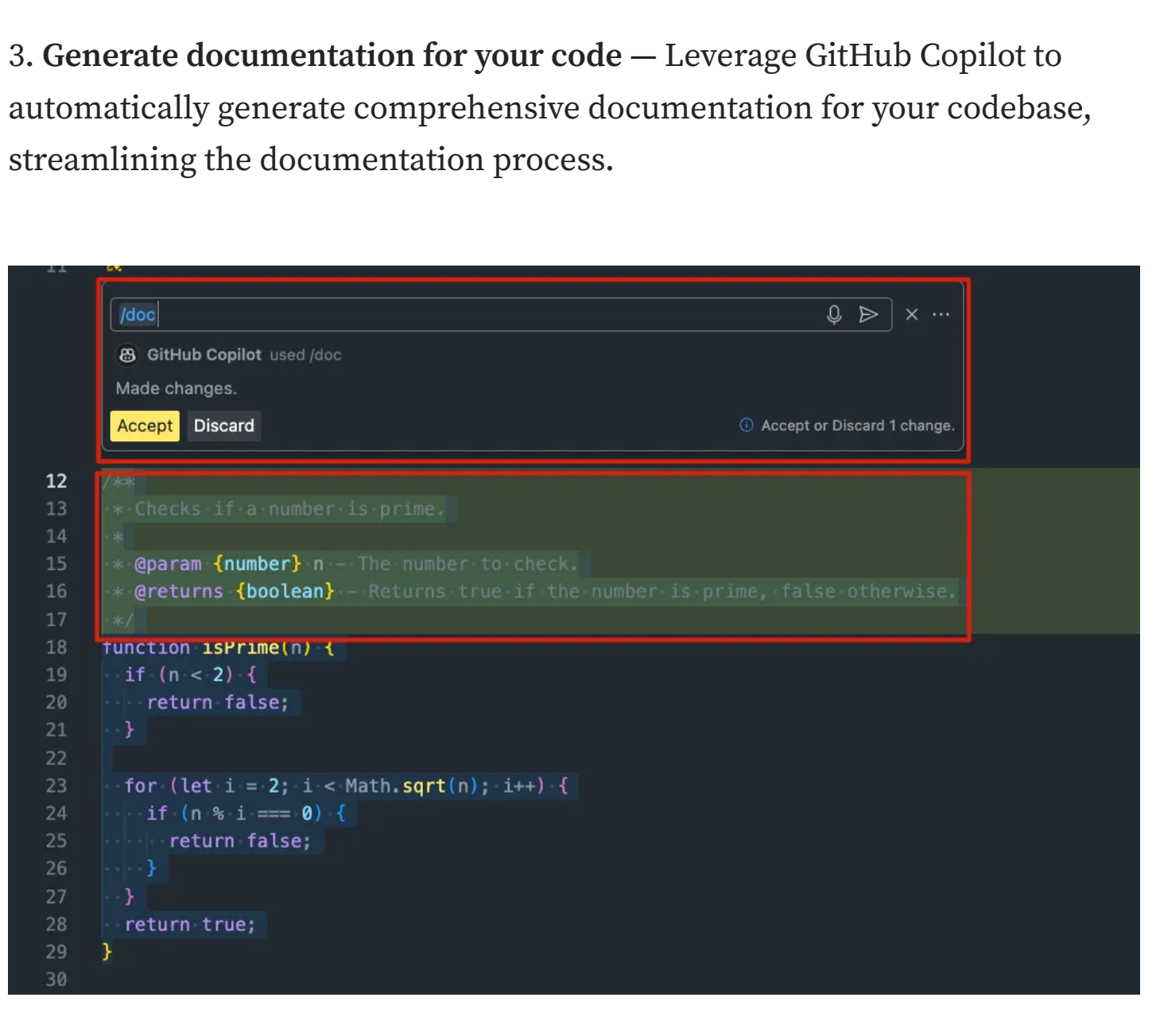
This usually happens when the input string doesn't have enough commas. In this case, it seems like the `line` variable doesn't contain two commas, which are needed to split it into three parts: `date_str`, `amount_str`, and `currency`.

You should check the value of `line` before it's split, and make sure it contains at least two commas. If it doesn't, you'll need to handle this case in your code to prevent the `ValueError` from being raised.

How can I handle the case when the input string doesn't have enough commas to split it into three parts?

```
ValueError: not enough values to unpack (expected 3, got 1)
```

3. **Generate documentation for your code** — Leverage GitHub Copilot to automatically generate comprehensive documentation for your codebase, streamlining the documentation process.

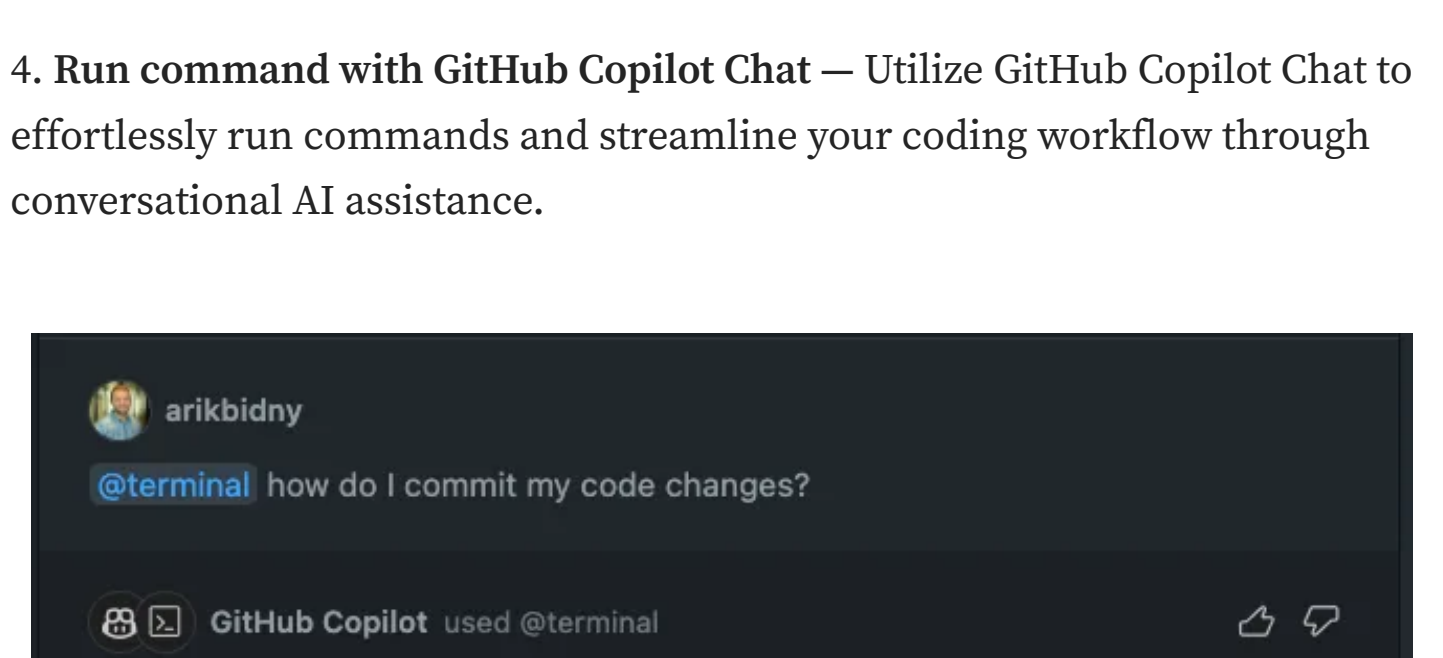


/doc

GitHub Copilot used /doc
Made changes.

```
/**
 * Checks if a number is prime.
 *
 * @param {number} n - The number to check.
 * @returns {boolean} - Returns true if the number is prime, false otherwise.
 */
function isPrime(n) {
  if (n < 2) {
    return false;
  }

  for (let i = 2; i < Math.sqrt(n); i++) {
    if (n % i === 0) {
      return false;
    }
  }
  return true;
}
```

4. **Run command with GitHub Copilot Chat** — Utilize GitHub Copilot Chat to effortlessly run commands and streamline your coding workflow through conversational AI assistance.



arikbidny
@terminal how do I commit my code changes?

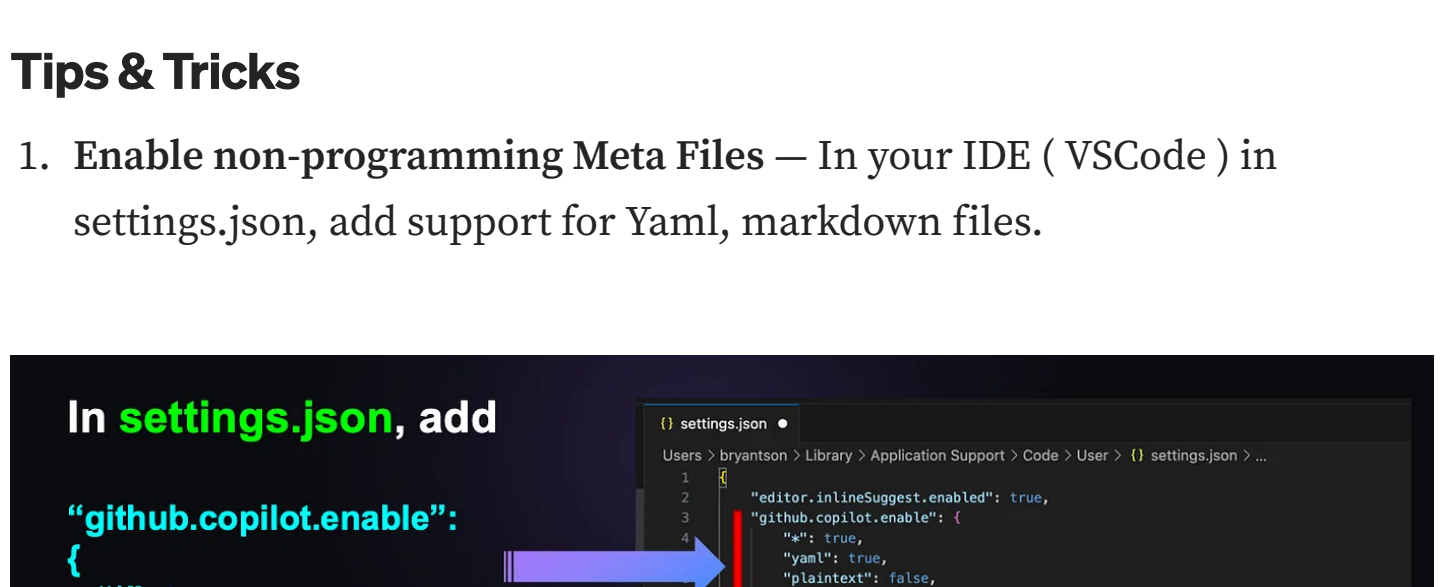
GitHub Copilot used @terminal

```
git add . && git commit -m "{commit_message}"
```

- `{commit_message}`: Replace with your commit message.

Tips & Tricks

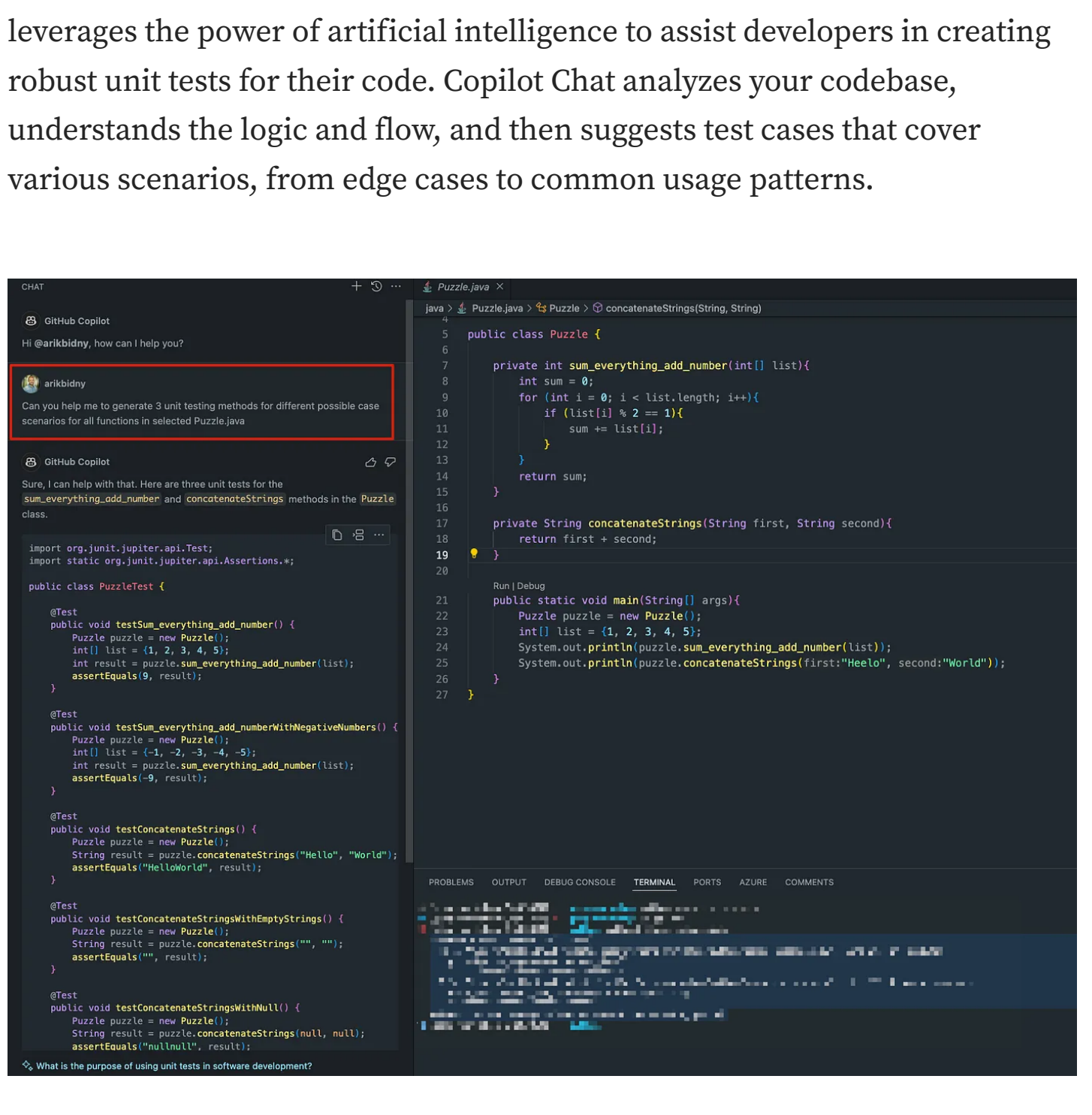
1. **Enable non-programming Meta Files** — In your IDE (VSCode) in settings.json, add support for Yaml, markdown files.



In settings.json, add

```
"github.copilot.enable":
{
  "*": true,
  "yaml": true,
  "plaintext": false,
  "markdown": true
}
```

2. **Unit testing generation using Copilot Chat** — This cutting-edge tool leverages the power of artificial intelligence to assist developers in creating robust unit tests for their code. Copilot Chat analyzes your codebase, understands the logic and flow, and then suggests test cases that cover various scenarios, from edge cases to common usage patterns.



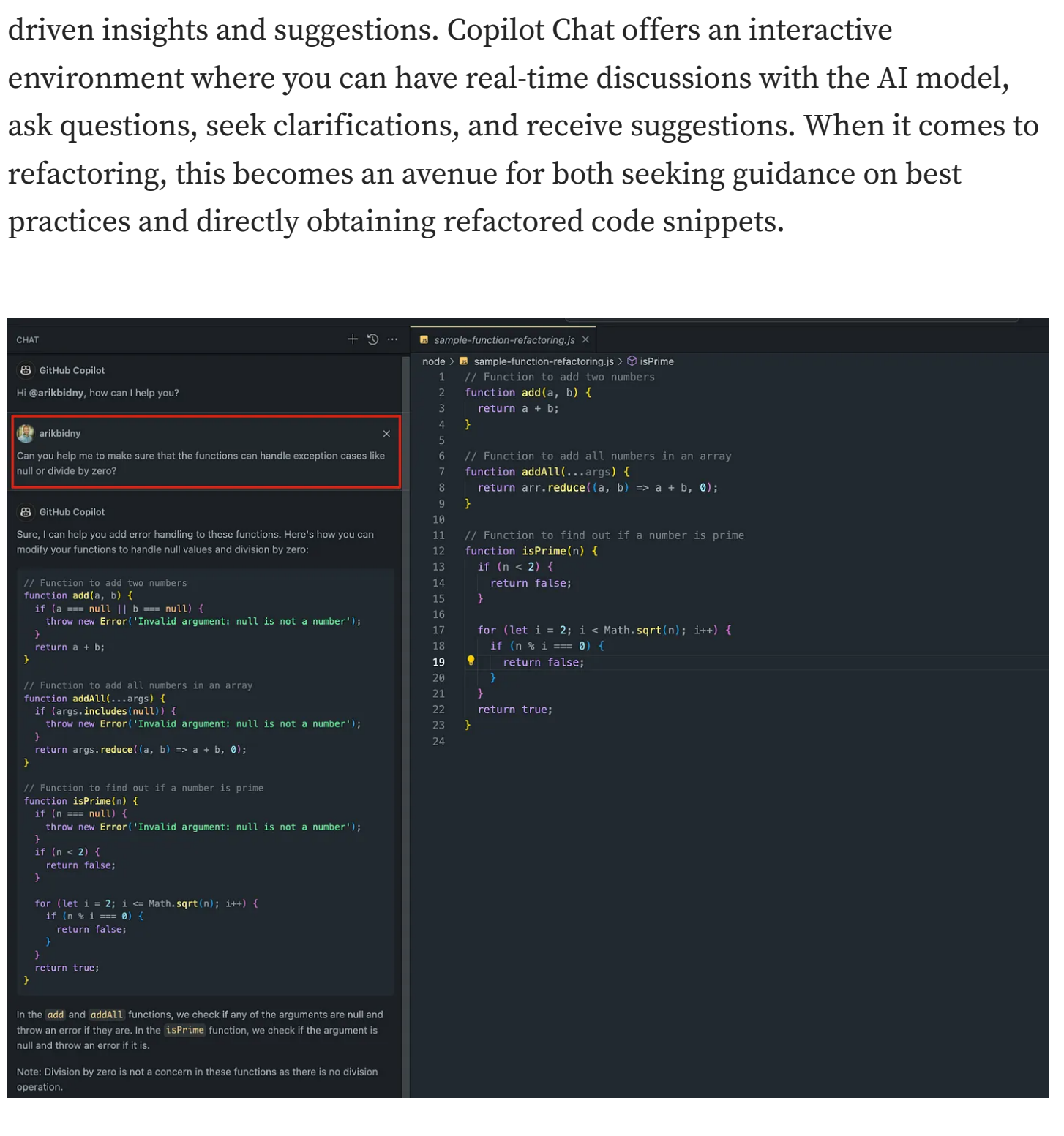
arikbidny
Can you help me to generate 3 unit testing methods for different possible case scenarios for all functions in selected Puzzle.java

GitHub Copilot
Sure, I can help with that. Here are three unit tests for the `sum_everything_add_number` and `concatenateStrings` methods in the `Puzzle` class.

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class PuzzleTest {

    @Test
    public void testSum_everything_add_number() {
        Puzzle puzzle = new Puzzle();
        int[] list = {1, 2, 3, 4, 5};
        int result = puzzle.sum_everything_add_number(list);
        assertEquals(9, result);
    }

    @Test
    public void testSum_everything_add_numberWithNegativeNumbers() {
        Puzzle puzzle = new Puzzle();
        int[] list = {-1, -2, -3, -4, -5};
        int result = puzzle.sum_everything_add_number(list);
        assertEquals(-9, result);
    }

    @Test
    public void testConcatenateStrings() {
        Puzzle puzzle = new Puzzle();
        String result = puzzle.concatenateStrings("Hello", "World");
        assertEquals("HelloWorld", result);
    }

    @Test
    public void testConcatenateStringsWithEmptyStrings() {
        Puzzle puzzle = new Puzzle();
        String result = puzzle.concatenateStrings("", "");
        assertEquals("", result);
    }

    @Test
    public void testConcatenateStringsWithNull() {
        Puzzle puzzle = new Puzzle();
        String result = puzzle.concatenateStrings(null, null);
        assertEquals("nullnull", result);
```

What is the purpose of using unit tests in software development?

3. **Refactoring Code using Copilot Chat** — Refactoring is a critical aspect of software development. It involves modifying existing code to improve its structure, readability, or performance, without altering its external behavior. GitHub Copilot Chat provides a new dimension to refactoring by offering AI-driven insights and suggestions. Copilot Chat offers an interactive environment where you can have real-time discussions with the AI model, ask questions, seek clarifications, and receive suggestions. When it comes to refactoring, this becomes an avenue for both seeking guidance on best practices and directly obtaining refactored code snippets.



arikbidny
Can you help me to make sure that the functions can handle exception cases like null or divide by zero?

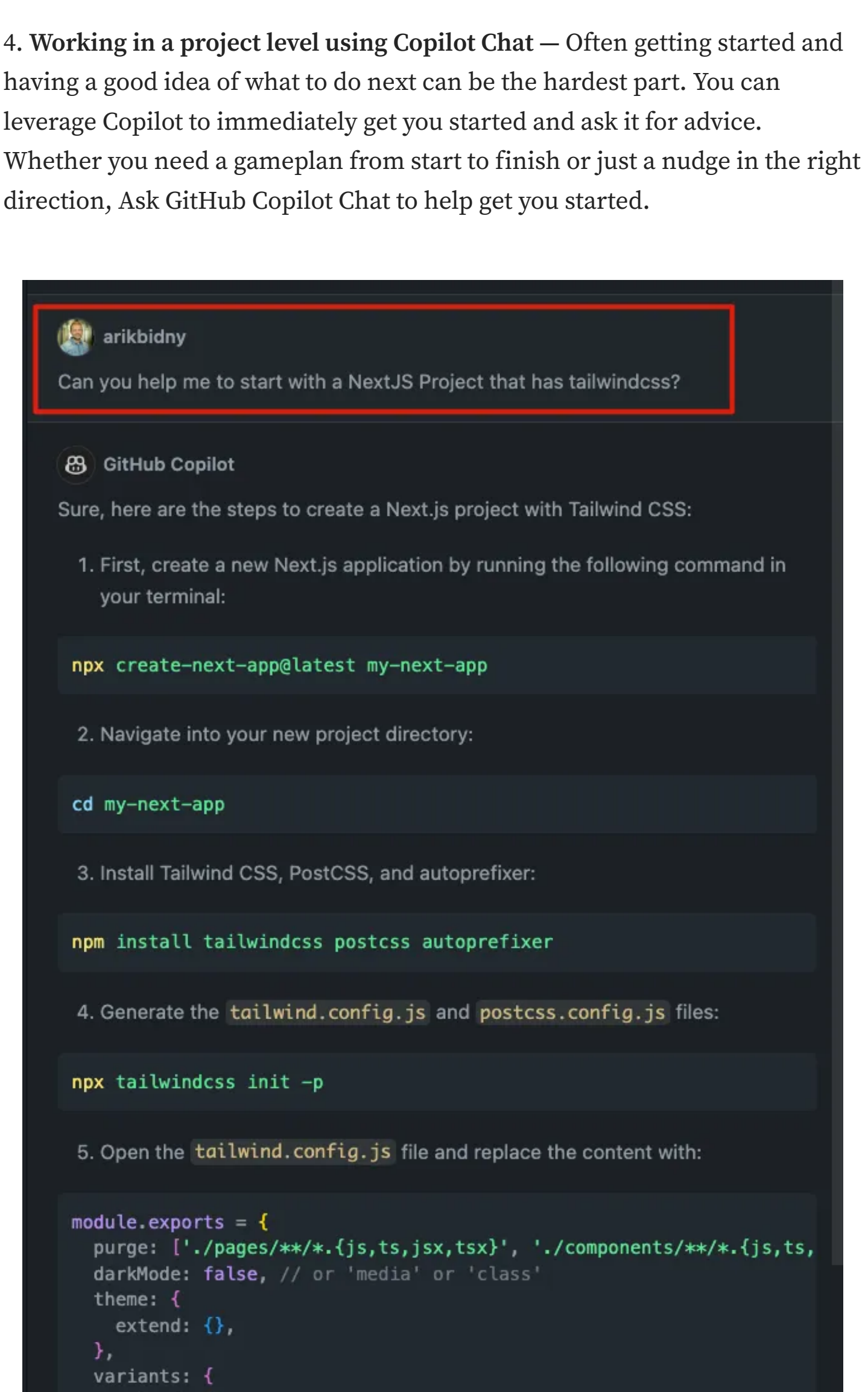
GitHub Copilot
Sure, I can help you add error handling to these functions. Here's how you can modify your functions to handle null values and division by zero:

```
// Function to add two numbers
function add(a, b) {
  if (a === null || b === null) {
    throw new Error('Invalid argument: null is not a number');
  }
  return a + b;
}

// Function to add all numbers in an array
function addAll(...args) {
  if (args.includes(null)) {
    throw new Error('Invalid argument: null is not a number');
  }
  return args.reduce((a, b) => a + b, 0);
}

// Function to find out if a number is prime
function isPrime(n) {
  if (n === null) {
    throw new Error('Invalid argument: null is not a number');
  }
  if (n < 2) {
    return false;
  }

  for (let i = 2; i <= Math.sqrt(n); i++) {
    if (n % i === 0) {
      return false;
    }
  }
  return true;
}
```

In the `add` and `addAll` functions, we check if any of the arguments are null and throw an error if they are. In the `isPrime` function, we check if the argument is null and throw an error if it is.

Note: Division by zero is not a concern in these functions as there is no division operation.

4. **Working in a project level using Copilot Chat** — Often getting started and having a good idea of what to do next can be the hardest part. You can leverage Copilot to immediately get you started and ask it for advice. Whether you need a gameplan from start to finish or just a nudge in the right direction, Ask GitHub Copilot Chat to help get you started.



arikbidny
Can you help me to start with a NextJS Project that has tailwindcss?

GitHub Copilot
Sure, here are the steps to create a Next.js project with Tailwind CSS:

1. First, create a new Next.js application by running the following command in your terminal:

```
npx create-next-app@latest my-next-app
```

2. Navigate into your new project directory:

```
cd my-next-app
```

3. Install Tailwind CSS, PostCSS, and autoprefixer:

```
npm install tailwindcss postcss autoprefixer
```

4. Generate the `tailwind.config.js` and `postcss.config.js` files:

```
npx tailwindcss init -p
```

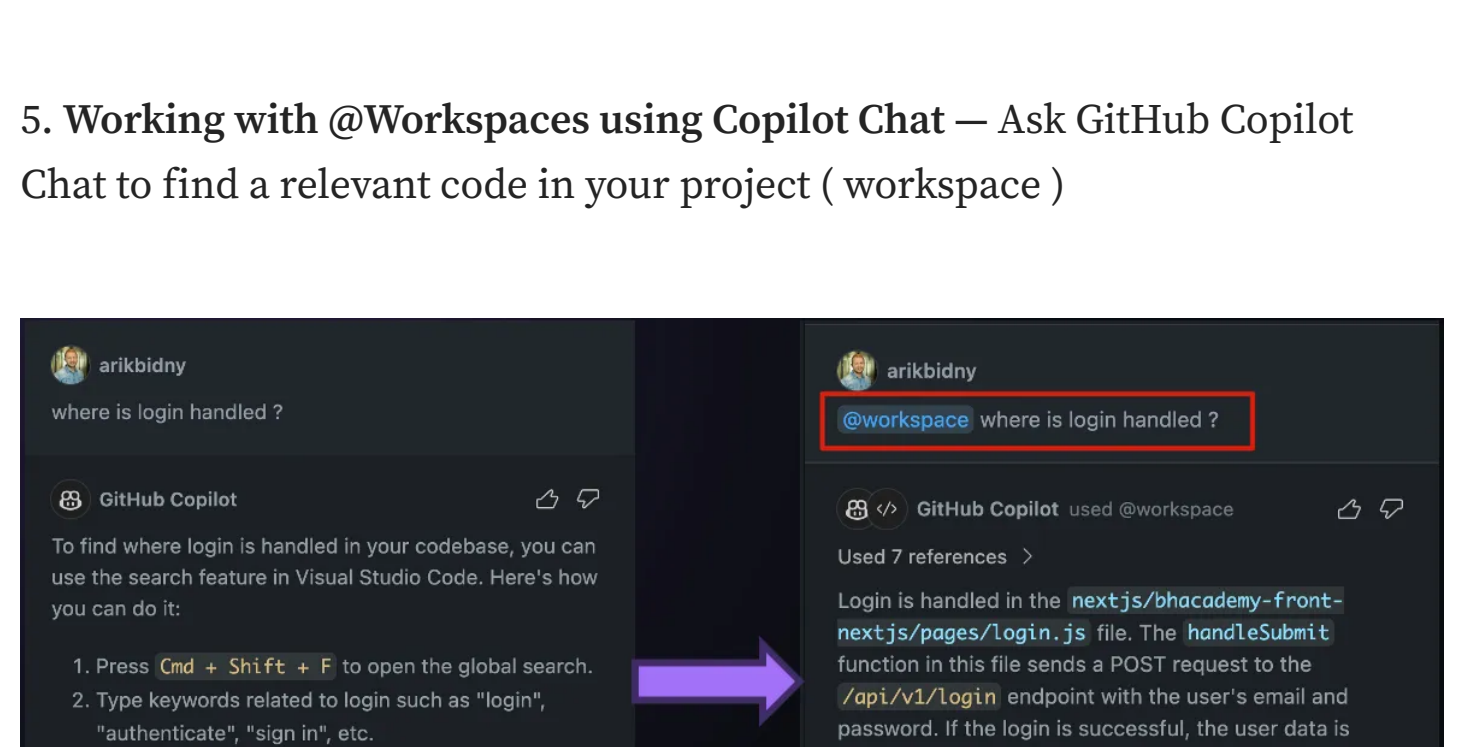
5. Open the `tailwind.config.js` file and replace the content with:

```
module.exports = {
  purge: ['./pages/**/*.{js,ts,jsx,tsx}', './components/**/*.{js,ts,
  darkMode: false, // or 'media' or 'class'
  theme: {
    extend: {},
  },
  variants: {
    extend: {},
  },
  plugins: [],
}
```

6. Open the `postcss.config.js` file and replace the content with:

```
module.exports = {
  plugins: {
    tailwindcss: {},
```

5. **Working with @Workspaces using Copilot Chat** — Ask GitHub Copilot Chat to find a relevant code in your project (workspace)



arikbidny
where is login handled ?

GitHub Copilot
To find where login is handled in your codebase, you can use the search feature in Visual Studio Code. Here's how you can do it:

1. Press `Cmd + Shift + F` to open the global search.
2. Type keywords related to login such as "login", "authenticate", "sign in", etc.
3. Press `Enter` to start the search.

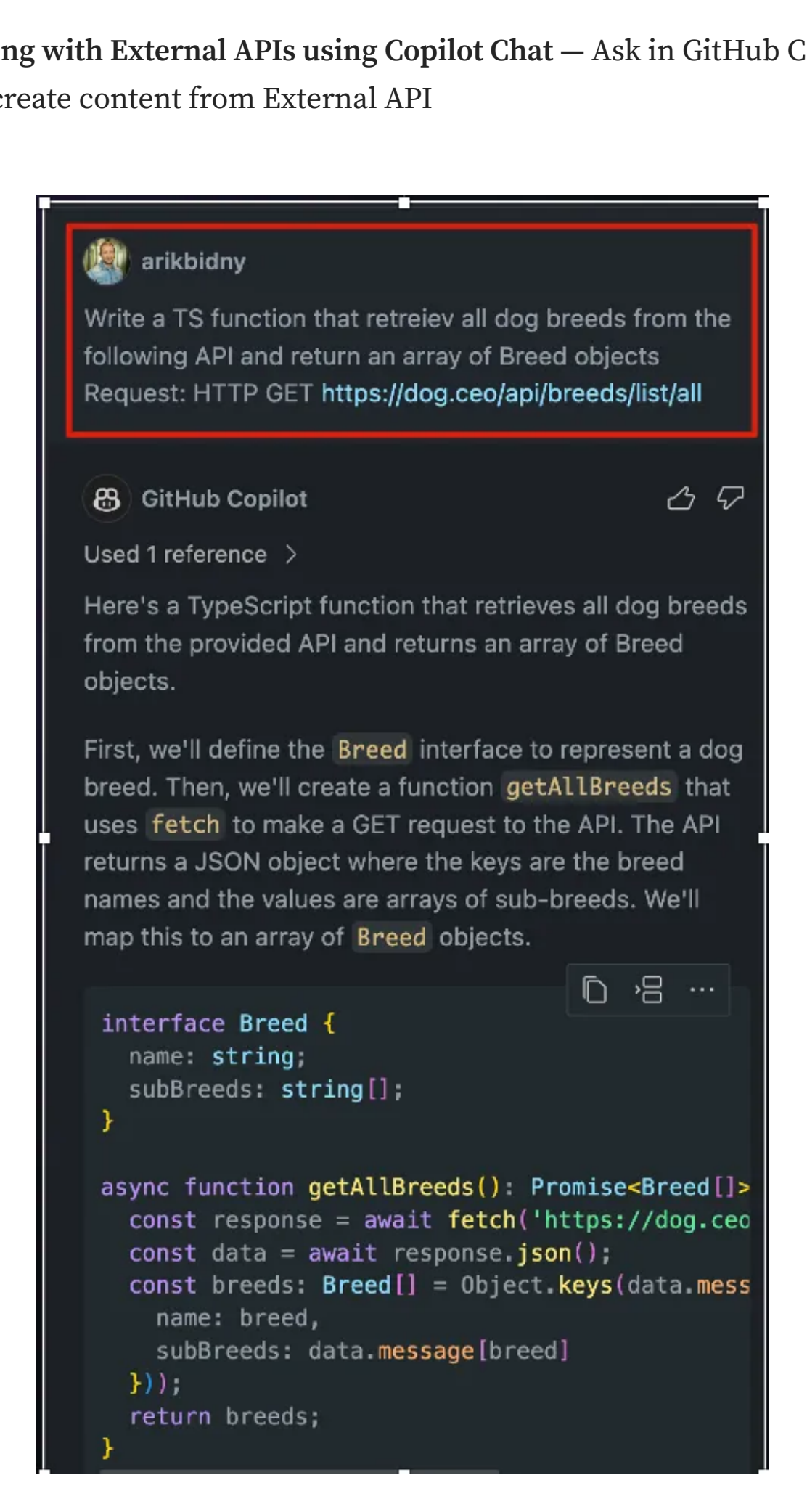
This will show you all the places in your code where these keywords are used. You can then navigate to those files and inspect the code to find where login is handled.

arikbidny
@workspace where is login handled ?

GitHub Copilot used @workspace
Used 7 references

Login is handled in the `nextjs/bhacademy-front-nextjs/pages/login.js` file. The `handleSubmit` function in this file sends a POST request to the `/api/v1/login` endpoint with the user's email and password. If the login is successful, the user data is stored in local storage and the user is redirected to the home page.

6. **Working with External APIs using Copilot Chat** — Ask in GitHub Copilot Chat to create content from External API



arikbidny
Write a TS function that retreiev all dog breeds from the following API and return an array of Breed objects
Request: HTTP GET https://dog.ceo/api/breeds/list/all

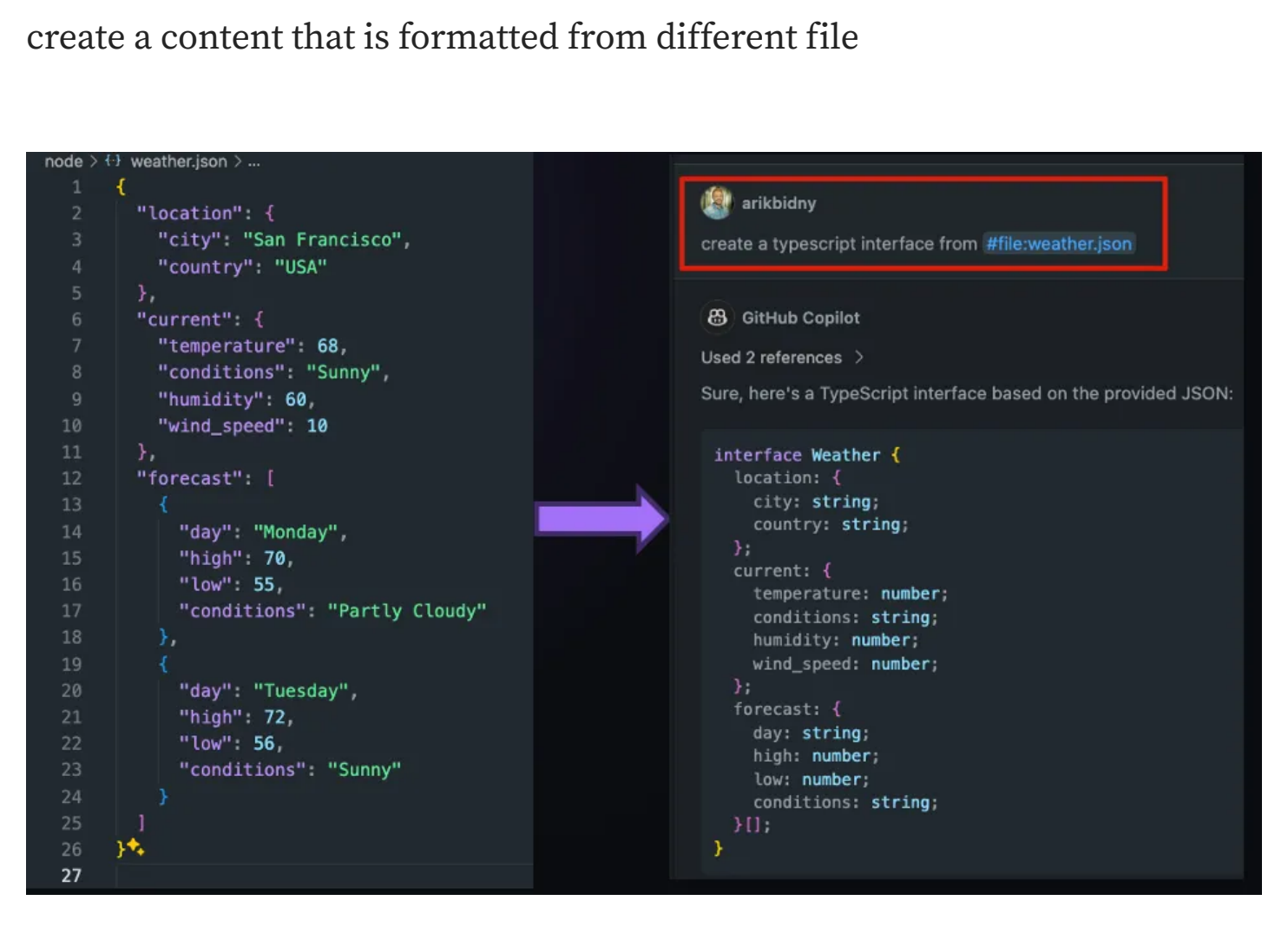
GitHub Copilot
Used 1 reference

Here's a TypeScript function that retrieves all dog breeds from the provided API and returns an array of Breed objects.

First, we'll define the `Breed` interface to represent a dog breed. Then, we'll create a function `getAllBreeds` that uses `fetch` to make a GET request to the API. The API returns a JSON object where the keys are the breed names and the values are arrays of sub-breeds. We'll map this to an array of `Breed` objects.

```
interface Breed {
  name: string;
  subBreeds: string[];
}

async function getAllBreeds(): Promise<Breed[]>
  const response = await fetch('https://dog.ceo
  const data = await response.json();
  const breeds: Breed[] = Object.keys(data.mess
    name: breed,
    subBreeds: data.message[breed]
  }));
  return breeds;
}
```

7. **Working with #file using Copilot Chat** — Ask in GitHub Copilot Chat to create a content that is formatted from different file



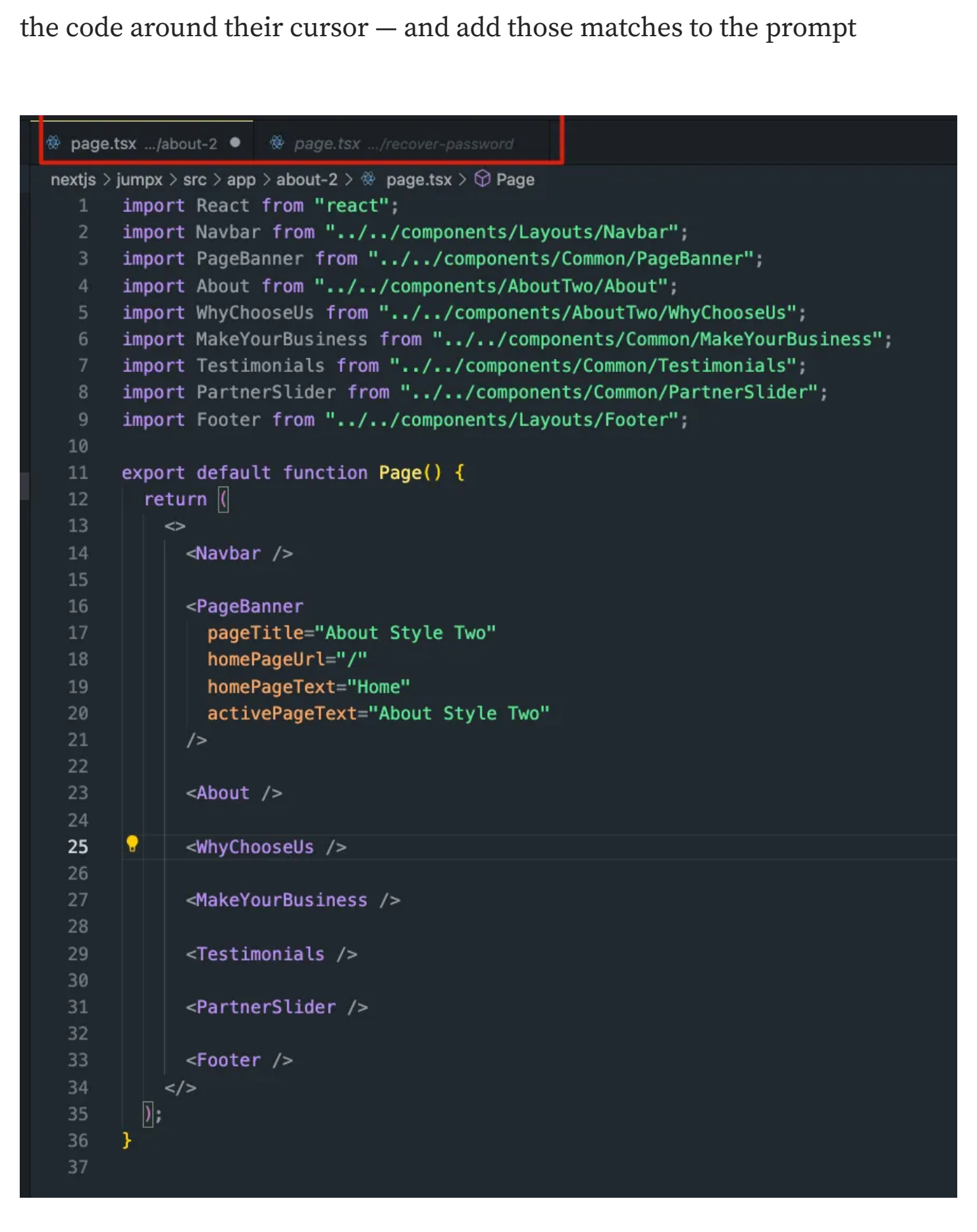
```
{
  "location": {
    "city": "San Francisco",
    "country": "USA"
  },
  "current": {
    "temperature": 68,
    "conditions": "Sunny",
    "humidity": 60,
    "wind_speed": 10
  },
  "forecast": [
    {
      "day": "Monday",
      "high": 70,
      "low": 55,
      "conditions": "Partly Cloudy"
    },
    {
      "day": "Tuesday",
      "high": 72,
      "low": 56,
      "conditions": "Sunny"
    }
  ]
}
```

arikbidny
create a typescript interface from #file:weather.json

GitHub Copilot
Used 2 references
Sure, here's a TypeScript interface based on the provided JSON:

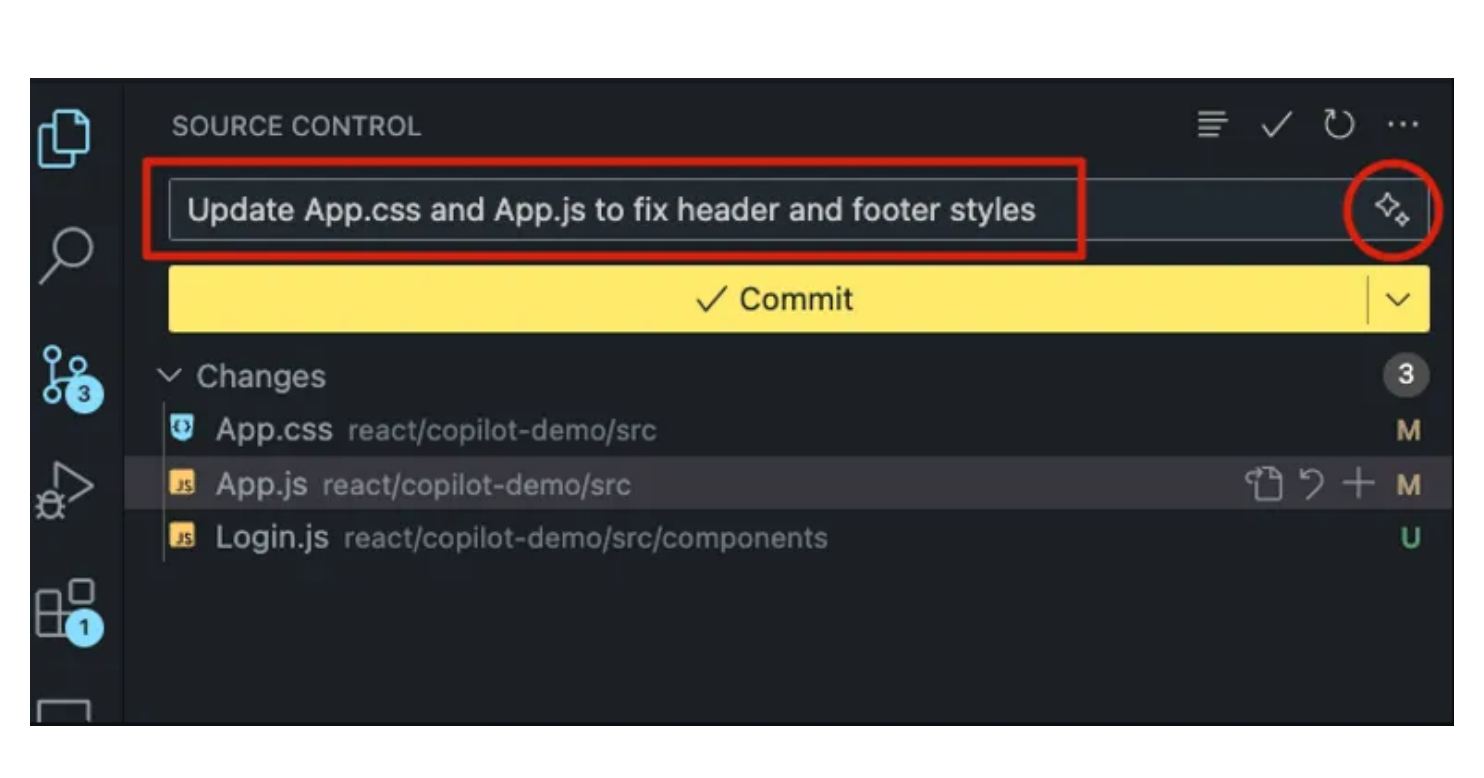
```
interface Weather {
  location: {
    city: string;
    country: string;
  };
  current: {
    temperature: number;
    conditions: string;
    humidity: number;
    wind_speed: number;
  };
  forecast: {
    day: string;
    high: number;
    low: number;
    conditions: string;
  }[];
}
```

8. **Neighboring Tabs** — Neighboring Tabs is the technique that allows GitHub Copilot to process all of the files open in a developer's IDE instead of just the single one the developer is working on. By opening all files relevant to their project, developers automatically invoke GitHub Copilot to comb through all of the data and find matching pieces of code between their open files and the code around their cursor — and add those matches to the prompt



```
import React from "react";
import Navbar from "../../components/Layouts/Navbar";
import PageBanner from "../../components/Common/PageBanner";
import About from "../../components/AboutTwo/About";
import WhyChooseUs from "../../components/AboutTwo/WhyChooseUs";
import MakeYourBusiness from "../../components/Common/MakeYourBusiness";
import Testimonials from "../../components/Common/Testimonials";
import PartnerSlider from "../../components/Common/PartnerSlider";
import Footer from "../../components/Layouts/Footer";

export default function Page() {
  return (
    <>
      <Navbar />

      <PageBanner
        pageTitle="About Style Two"
        homePageUrl="/"
        homePageText="Home"
        activePageText="About Style Two"
      />

      <About />

      <WhyChooseUs />

      <MakeYourBusiness />

      <Testimonials />

      <PartnerSlider />

      <Footer />
    </>
  );
}
```

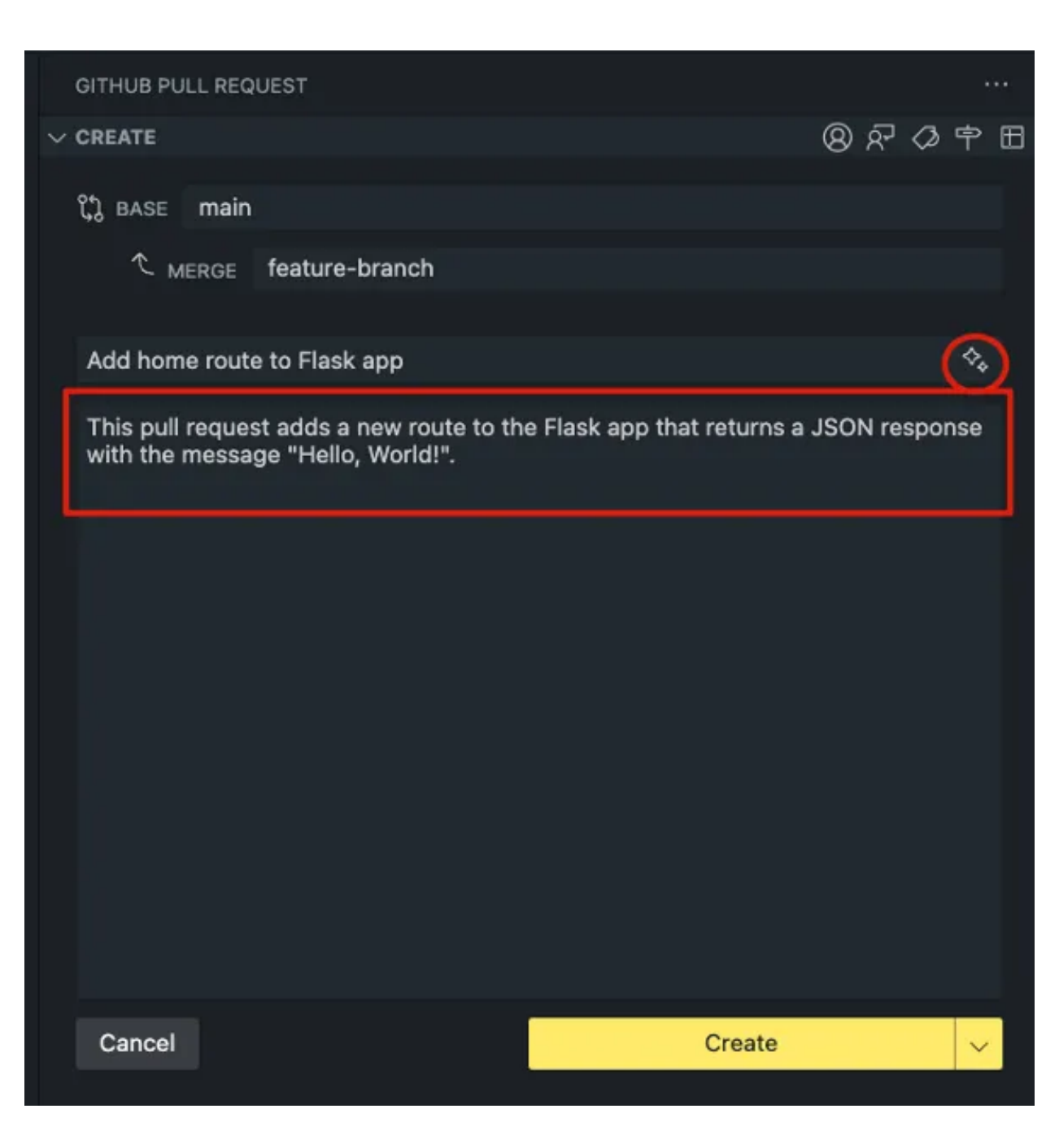
9. **Commit Changes using GitHub Copilot** — You can use GitHub Copilot from the IDE to create a commit message for you.



SOURCE CONTROL
Update App.css and App.js to fix header and footer styles

✓ Commit
Changes
App.css react/copilot-demo/src
App.js react/copilot-demo/src
Login.js react/copilot-demo/src/components

10. **Generate Pull Request messages using Copilot** — Generate a pull request message from your IDE, Please note that in order to use this feature, you will need to install an additional extension called 'GitHub Pull Requests'.

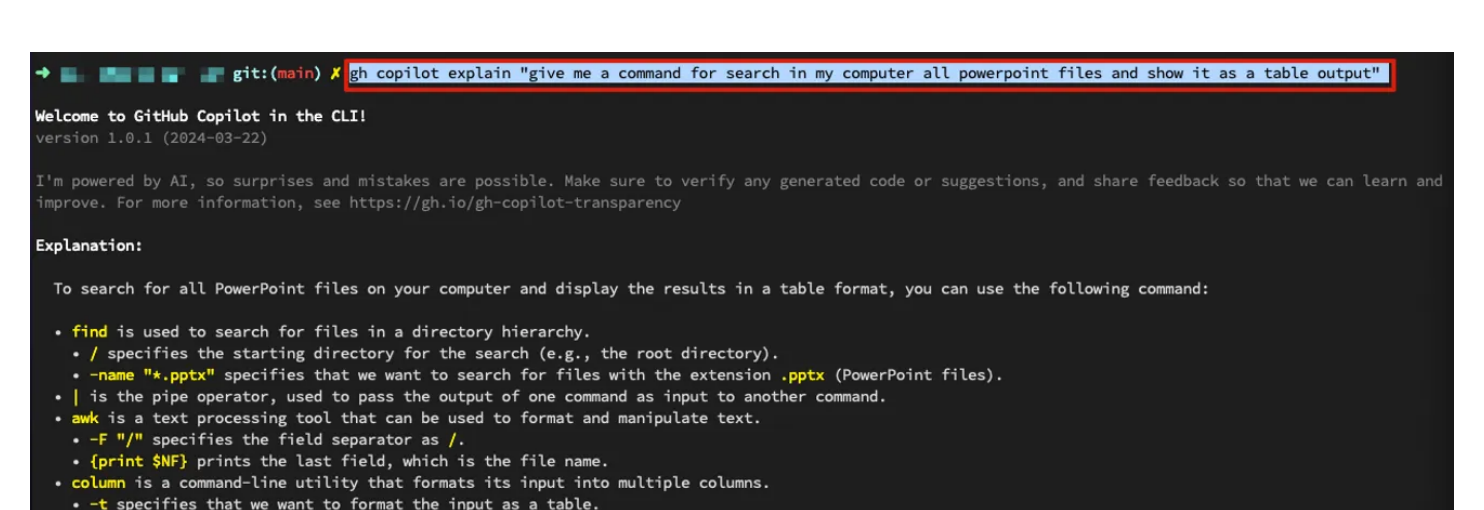


Add home route to Flask app

This pull request adds a new route to the Flask app that returns a JSON response with the message "Hello, World!".

11. **Working with GitHub Copilot CLI** —
GitHub Copilot CLI is a command-line interface that extends GitHub's AI-powered code suggestions to the terminal. It allows developers to access code completions, explanations, and scripting capabilities directly in their command-line environment, making it ideal for those who prefer terminal-based workflows or need to integrate AI assistance into automated scripts.

How to install GitHub Copilot CLI — https://docs.github.com/en/copilot/github-copilot-in-the-cli/setting-up-github-copilot-in-the-cli



```
git:(main) ✗ gh copilot explain "give me a command for search in my computer all powerpoint files and show it as a table output"

Welcome to GitHub Copilot in the CLI!
version 1.0.1 (2024-03-22)

I'm powered by AI, so surprises and mistakes are possible. Make sure to verify any generated code or suggestions, and share feedback so that we can learn and improve. For more information, see https://gh.io/gh-copilot-transparency

Explanation:

  To search for all PowerPoint files on your computer and display the results in a table format, you can use the following command:

  • find is used to search for files in a directory hierarchy.
    • / specifies the starting directory for the search (e.g., the root directory).
    • -name "*.pptx" specifies that we want to search for files with the extension .pptx (PowerPoint files).
  • | is the pipe operator, used to pass the output of one command as input to another command.
  • awk is a text processing tool that can be used to format and manipulate text.
    • -F "/" specifies the field separator as /.
    • {print $NF} prints the last field, which is the file name.
  • column is a command-line utility that formats its input into multiple columns.
    • -t specifies that we want to format the input as a table.
```

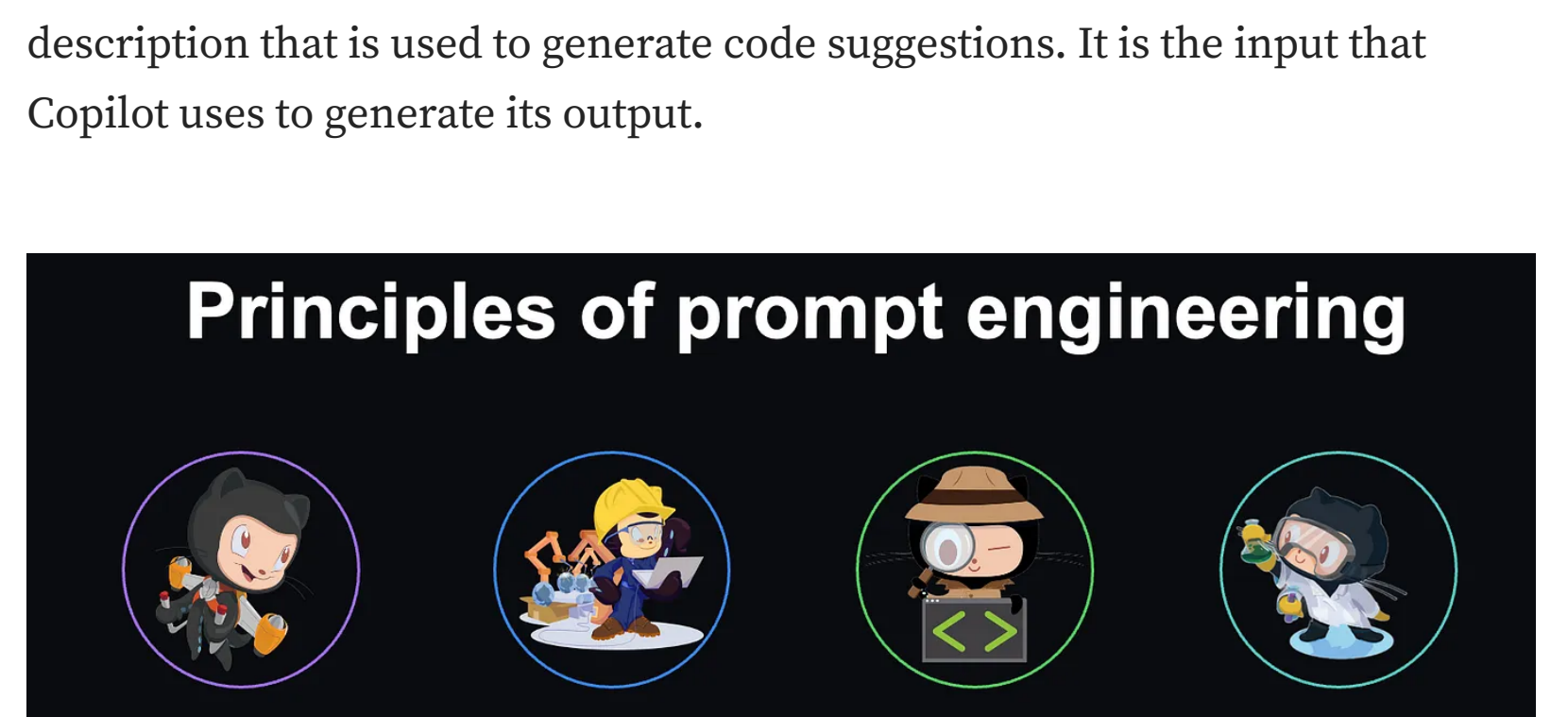
```
Putting it all together, the command would be:

find / -name "*.pptx" | awk -F "/" '{print $NF}' | column -t
```

12. **More Copilot Chat commands —**

> */explain — Explains how the selected code works*
>
> */tests — Generates unit tests for the selected code*
>
> */fix — Proposes a fix for the problems in the selected code*
>
> */workspace — Ask a question about the files in your current workspace*
>
> */help — Learn how to use Copilot Chat*

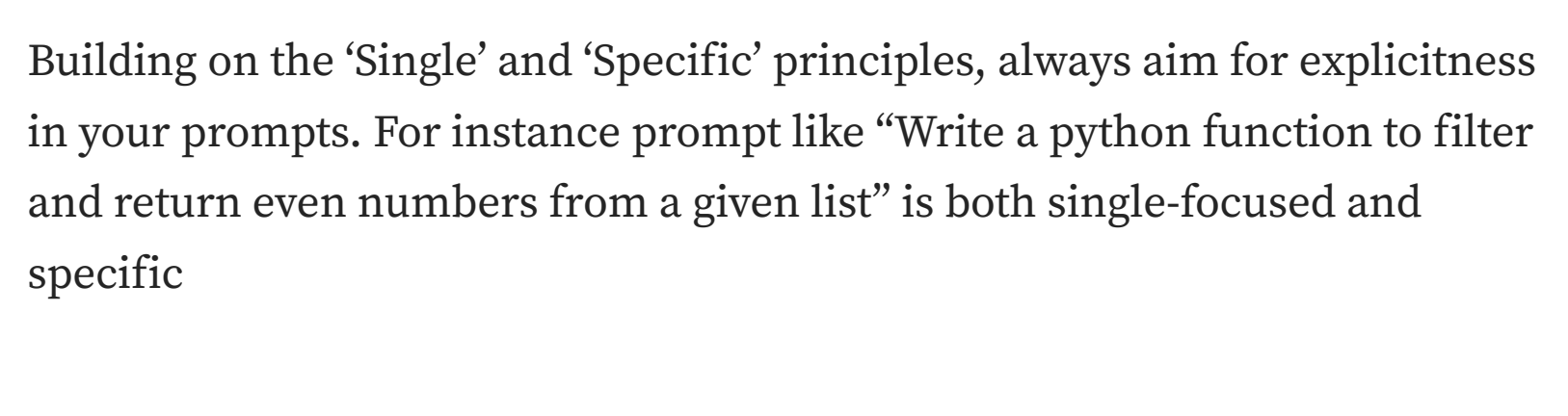
Prompt Engineering

In the context of Copilot, a prompt is a piece of code or natural language description that is used to generate code suggestions. It is the input that Copilot uses to generate its output.


Principles of prompt engineering
Single
Always focus your prompt on a single, well-defined task or question.
Specifc
Ensure that your instructions are explicit and detailed
Short
While being specific, keep prompts concise and to the point
Surround
Utilize descriptive filenames and keep related files open

Provide enough clarity

Building on the 'Single' and 'Specific' principles, always aim for explicitness in your prompts. For instance prompt like "Write a python function to filter and return even numbers from a given list" is both single-focused and specific

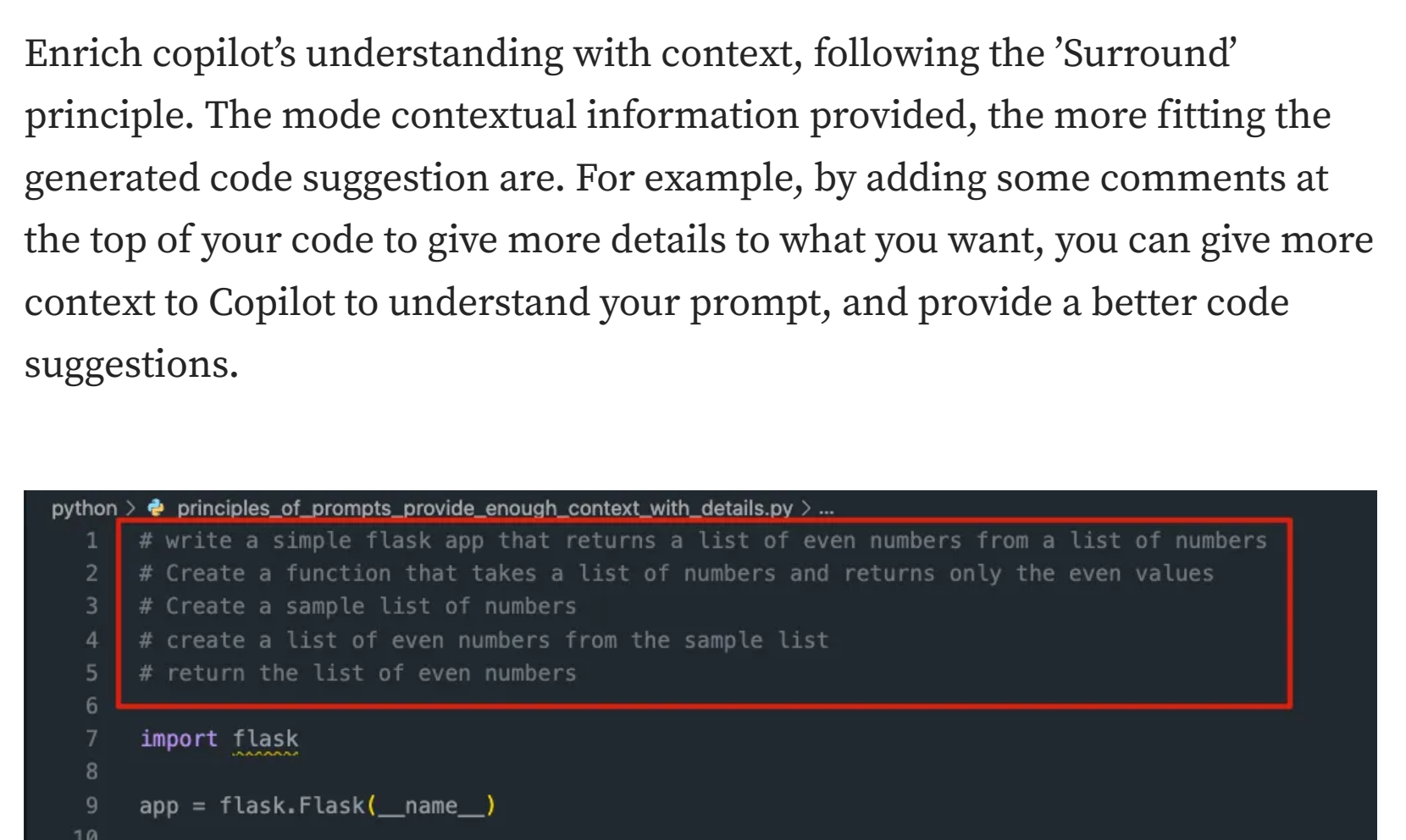


Write a Python function to filter and return even numbers from a given list.

```python
def filter_even_numbers(lst):
    return [num for num in lst if num % 2 == 0]
```

Provide enough context with details

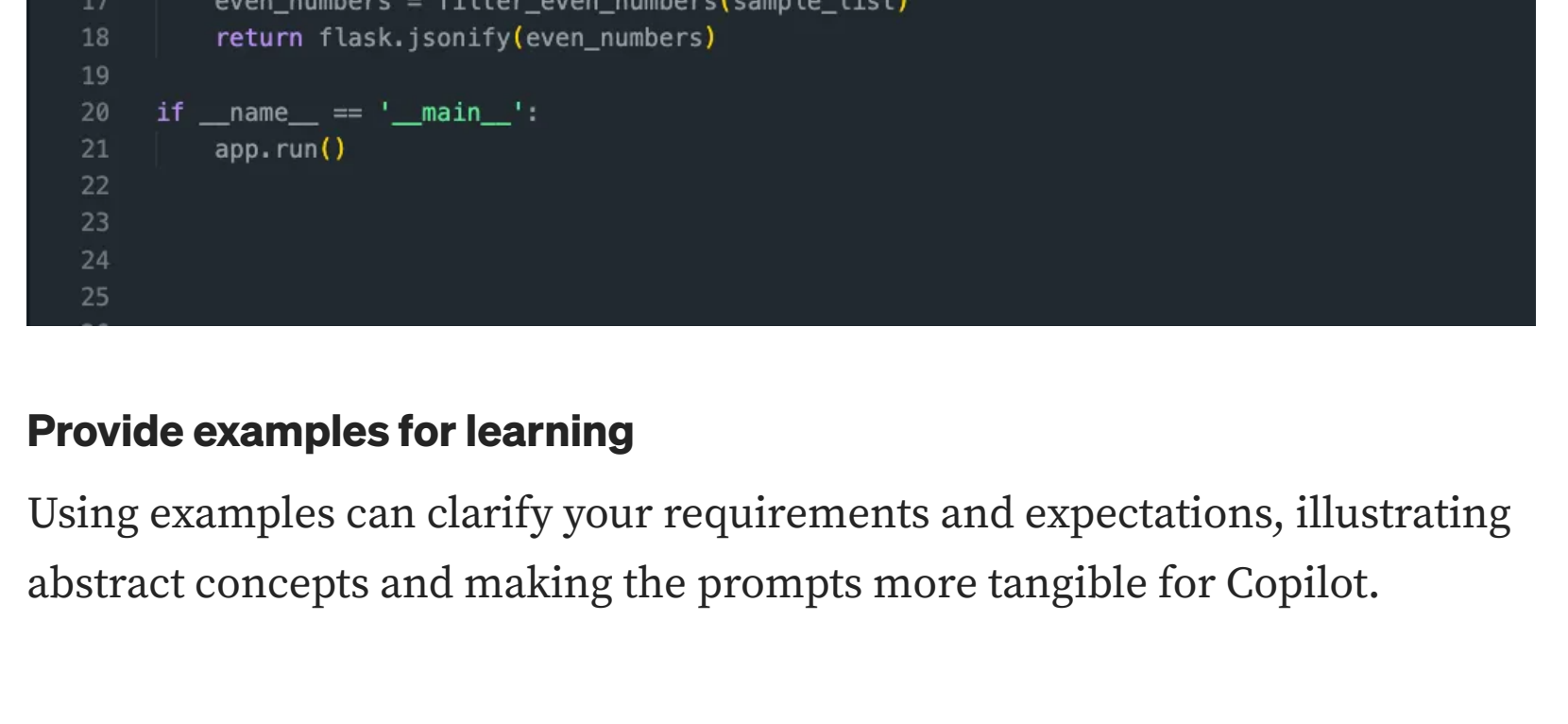
Enrich copilot's understanding with context, following the 'Surround' principle. The mode contextual information provided, the more fitting the generated code suggestion are. For example, by adding some comments at the top of your code to give more details to what you want, you can give more context to Copilot to understand your prompt, and provide a better code suggestions.


python > principles_of_prompts_provide_enough_context_with_details.py > ...

```python
# write a simple flask app that returns a list of even numbers from a list of numbers
# Create a function that takes a list of numbers and returns only the even values
# Create a sample list of numbers
# create a list of even numbers from the sample list
# return the list of even numbers

import flask

app = flask.Flask(__name__)

def filter_even_numbers(lst):
    return [num for num in lst if num % 2 == 0]

@app.route('/')
def index():
    sample_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    even_numbers = filter_even_numbers(sample_list)
    return flask.jsonify(even_numbers)

if __name__ == '__main__':
    app.run()
```

Provide examples for learning

Using examples can clarify your requirements and expectations, illustrating abstract concepts and making the prompts more tangible for Copilot.


python > principles_of_prompts_provide_example_for_learning.py > ...

```python
# create a sample list of numbers
# create a list of even numbers from the sample list
# return the list of even numbers
# Example: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] => [2, 4, 6, 8, 10]


list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_numbers = [num for num in list if num % 2 == 0]
print(even_numbers)
```

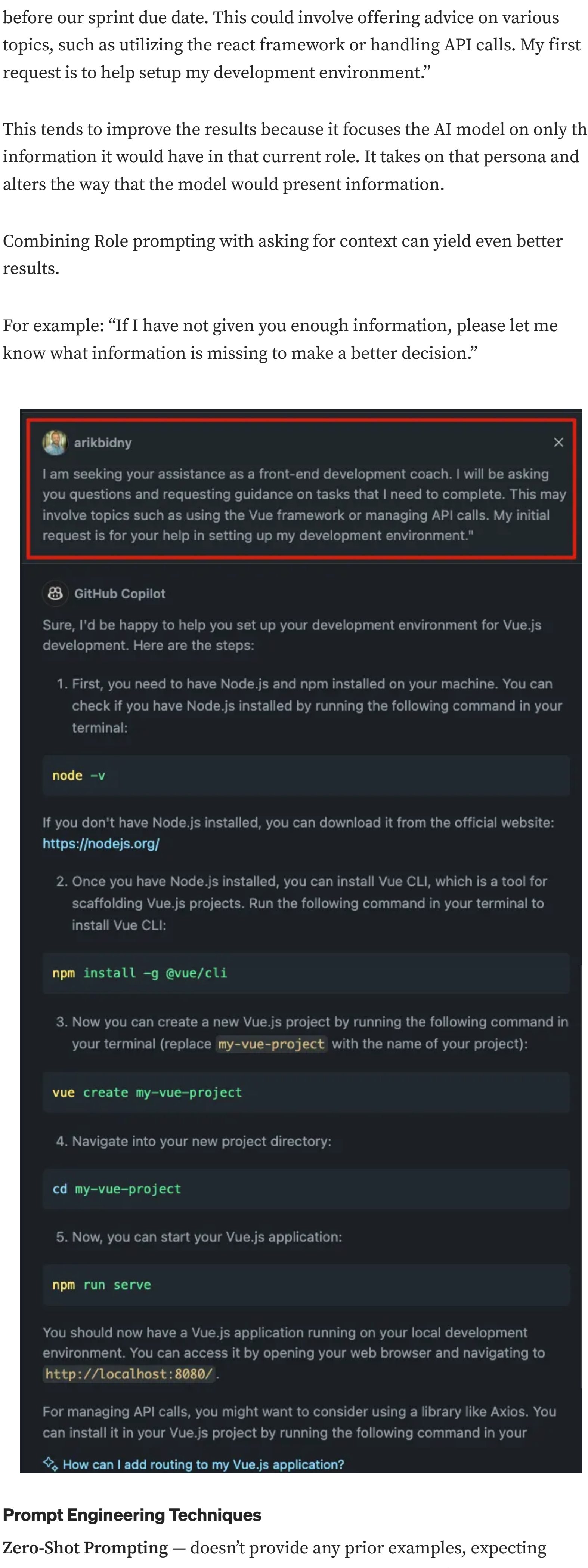
Role Prompting

Like every llm application we can to define a "system prompt", If you take a look at the example, here we have a prompt that states "I want you to act as a front end developer coach. I will provide some questions and the task that I have to complete and would like for you to walk me through the process before our sprint due date. This could involve offering advice on various topics, such as utilizing the react framework or handling API calls. My first request is to help setup my development environment."

This tends to improve the results because it focuses the AI model on only the information it would have in that current role. It takes on that persona and alters the way that the model would present information.

Combining Role prompting with asking for context can yield even better results.

For example: "If I have not given you enough information, please let me know what information is missing to make a better decision."



arikbidny

I am seeking your assistance as a front-end development coach. I will be asking you questions and requesting guidance on tasks that I need to complete. This may involve topics such as using the Vue framework or managing API calls. My initial request is for your help in setting up my development environment."

GitHub Copilot

Sure, I'd be happy to help you set up your development environment for Vue.js development. Here are the steps:

1. First, you need to have Node.js and npm installed on your machine. You can check if you have Node.js installed by running the following command in your terminal:

```
node -v
```

If you don't have Node.js installed, you can download it from the official website: https://nodejs.org/

2. Once you have Node.js installed, you can install Vue CLI, which is a tool for scaffolding Vue.js projects. Run the following command in your terminal to install Vue CLI:

```
npm install -g @vue/cli
```

3. Now you can create a new Vue.js project by running the following command in your terminal (replace `my-vue-project` with the name of your project):

```
vue create my-vue-project
```

4. Navigate into your new project directory:

```
cd my-vue-project
```

5. Now, you can start your Vue.js application:

```
npm run serve
```

You should now have a Vue.js application running on your local development environment. You can access it by opening your web browser and navigating to `http://localhost:8080/`.

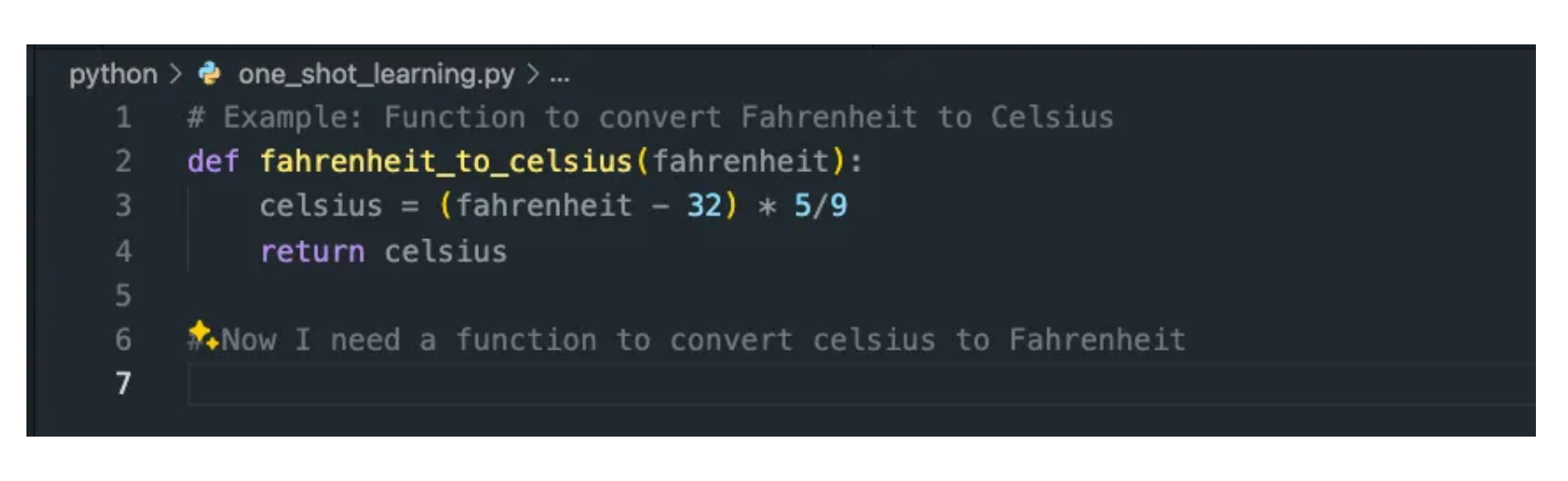
For managing API calls, you might want to consider using a library like Axios. You can install it in your Vue.js project by running the following command in your

How can I add routing to my Vue.js application?

Prompt Engineering Techniques

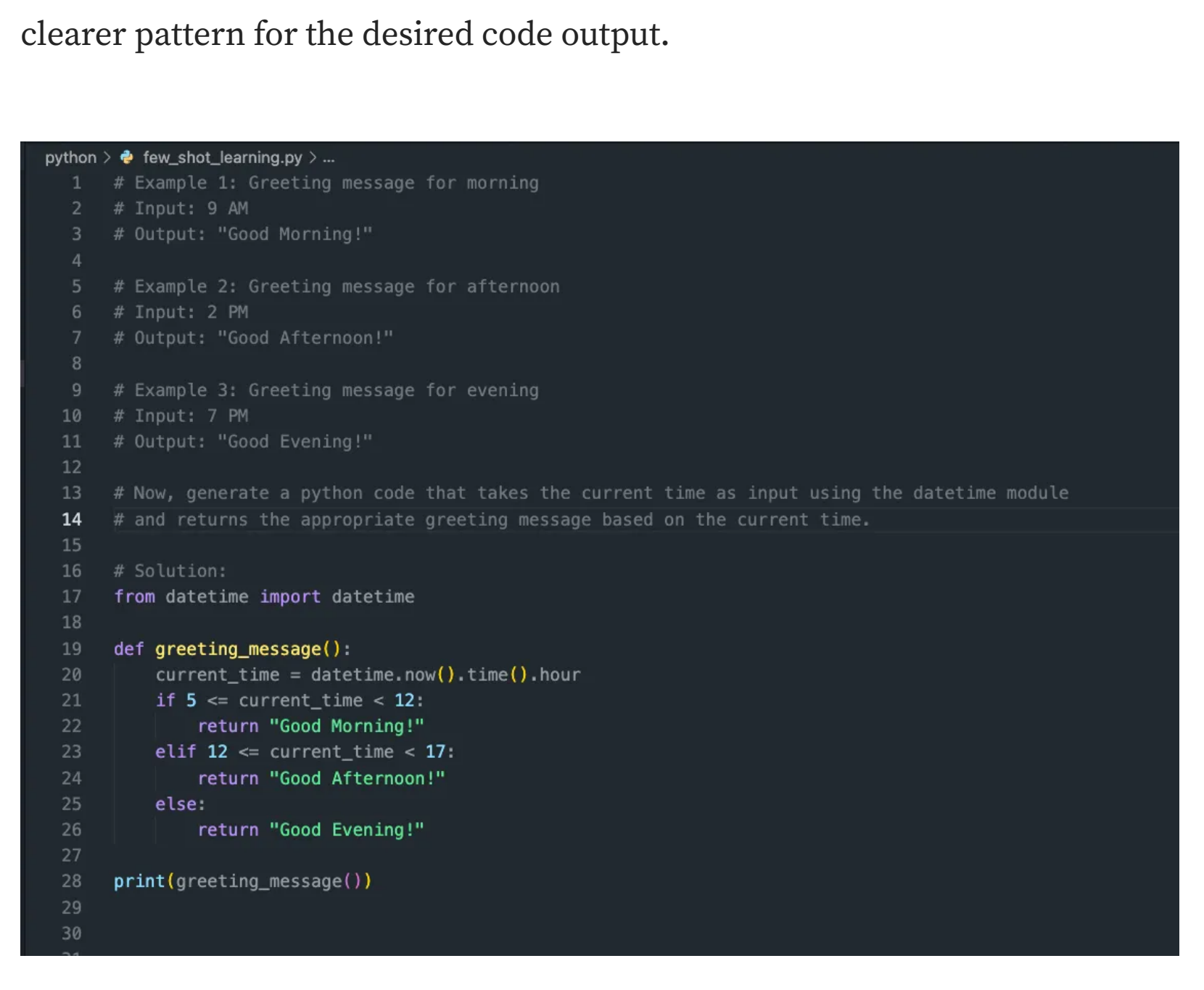
Zero-Shot Prompting — doesn't provide any prior examples, expecting Copilot to understand and generate relevant code purely from the given task description.

One-Shot Prompting — provides a single example to set the context, assisting Copilot in generating a similar outcome.



```python
# Example: Function to convert Fahrenheit to Celsius
def fahrenheit_to_celsius(fahrenheit):
    celsius = (fahrenheit - 32) * 5/9
    return celsius

#Now I need a function to convert celsius to Fahrenheit
```

Few-Shot Prompting — involves offering multiple examples to establish a clearer pattern for the desired code output.



```python
# Example 1: Greeting message for morning
# Input: 9 AM
# Output: "Good Morning!"

# Example 2: Greeting message for afternoon
# Input: 2 PM
# Output: "Good Afternoon!"

# Example 3: Greeting message for evening
# Input: 7 PM
# Output: "Good Evening!"

# Now, generate a python code that takes the current time as input using the datetime module
# and returns the appropriate greeting message based on the current time.

# Solution:
from datetime import datetime

def greeting_message():
    current_time = datetime.now().time().hour
    if 5 <= current_time < 12:
        return "Good Morning!"
    elif 12 <= current_time < 17:
        return "Good Afternoon!"
    else:
        return "Good Evening!"

print(greeting_message())
```

Conclusion

In conclusion, GitHub Copilot presents a paradigm shift in the way developers approach coding, offering a powerful blend of AI assistance and coding collaboration. By mastering prompt engineering, adhering to best practices, and exploring tips and tricks, developers can fully unlock the potential of Copilot, transforming it from a mere tool into an indispensable ally in their coding journey. With Copilot by their side, developers can not only write code faster but also write better code, ultimately enhancing productivity and innovation in the software development process. Embrace the future of coding with GitHub Copilot and watch your development workflow soar to new heights.

Github Copilot · Prompt Engineering Tips · Github Copilot Chat · Github · Microsoft

68

Written by Arik Bidny

26 Followers · 2 Following

Sr. Technology Specialist - Microsoft

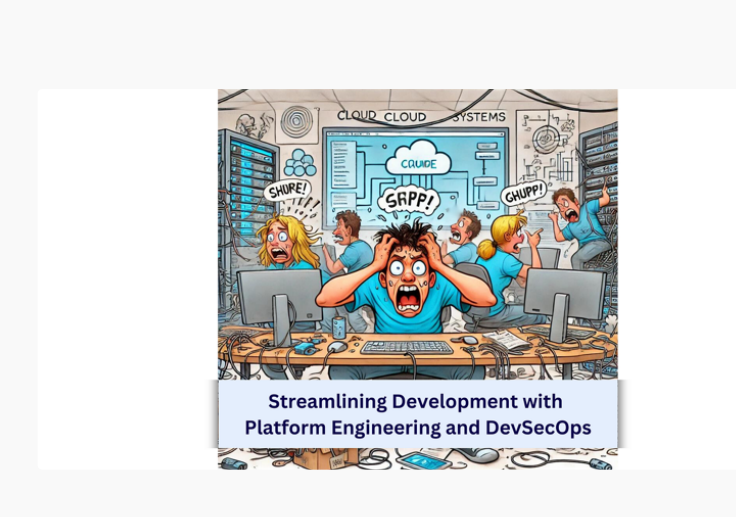
Follow

No responses yet

Till Meineke

What are your thoughts?

More from Arik Bidny

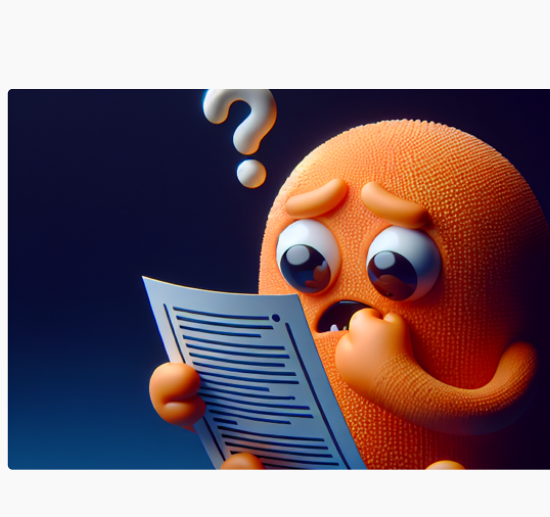

How to:
Build LLM App in 10 min

Arik Bidny

Build a Powerful LLM App in Just 10 Minutes: A Step-by-Step Guide

Creating an app, especially one powered by a Large Language Model (LLM), often feels lik...

Oct 1, 2024 · 54

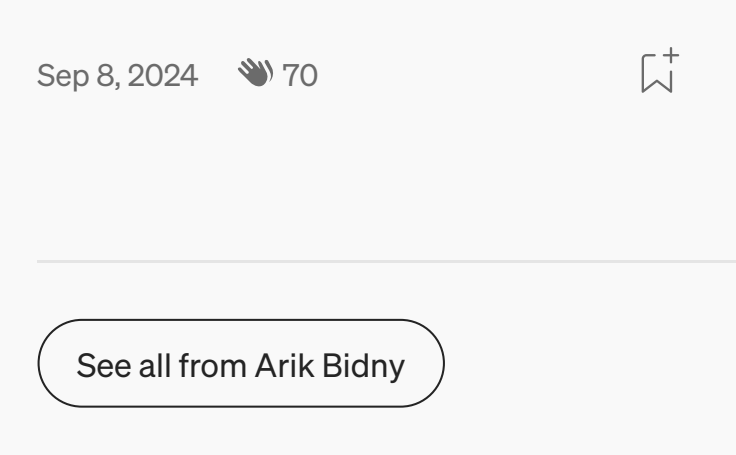


Arik Bidny

Wrap Azure Assistants API with NodeJS App

Assistants API, a new feature of Azure OpenAI Service, is now available in public...

Apr 19, 2024 · 59

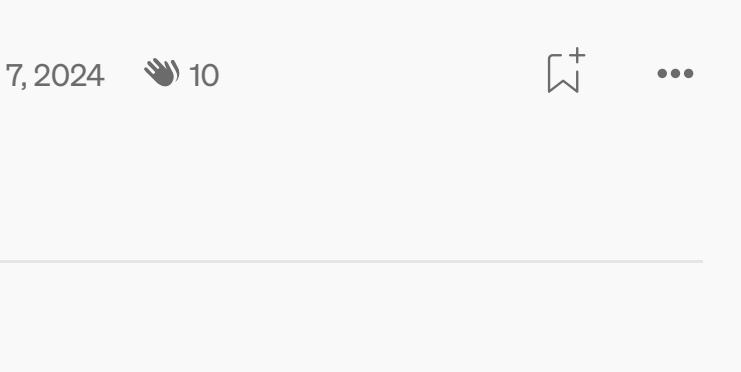

Streamlining Development with Platform Engineering and DevSecOps

Arik Bidny

Streamlining Development with Platform Engineering and...

Platform engineering is a practice rooted in DevOps principles, aimed at enhancing...

Sep 8, 2024 · 70



Arik Bidny

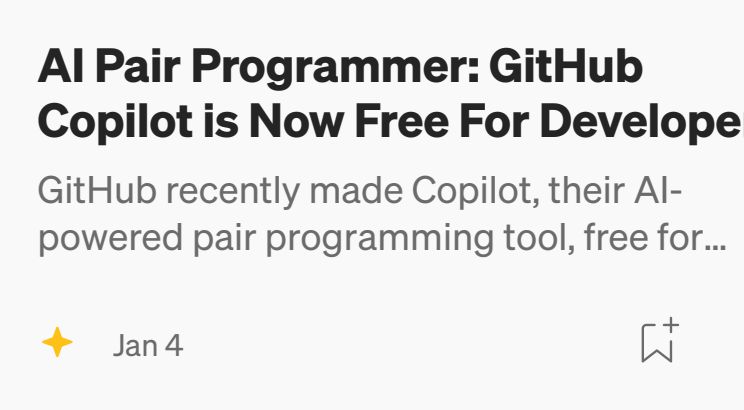
Enhancing Search Capabilities in RAG Applications Using Azure...

Retrieval-Augmented Generation (RAG) techniques significantly enhance search...

May 7, 2024 · 10

See all from Arik Bidny

Recommended from Medium

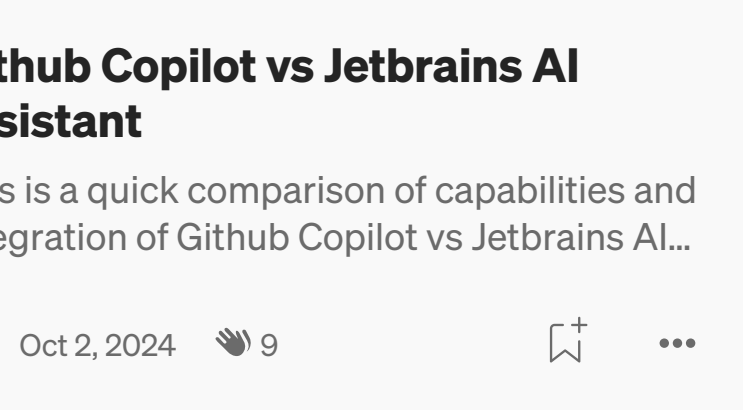


Ganesh Bajaj

AI Pair Programmer: GitHub Copilot is Now Free For Developers

GitHub recently made Copilot, their AI-powered pair programming tool, free for...

Jan 4



Enzo Lombardi

Github Copilot vs Jetbrains AI assistant

This is a quick comparison of capabilities and integration of Github Copilot vs Jetbrains AI...

Oct 2, 2024 · 9

Lists

Icon Design
39 stories · 504 saves

data science and AI
40 stories · 338 saves

Tech & Tools
23 stories · 402 saves

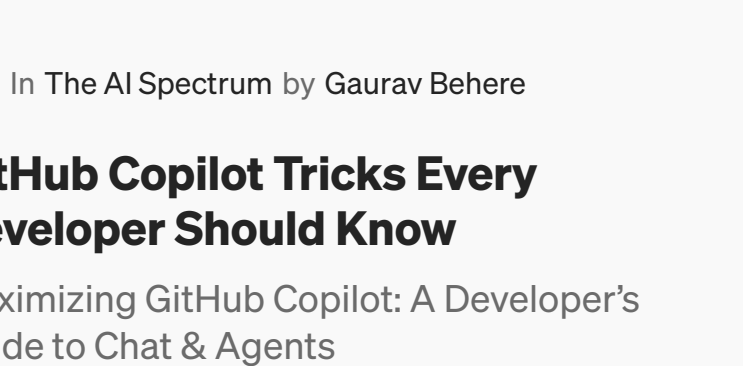
Generative AI Recommended Reading
52 stories · 1679 saves

Bobby

Sourcegraph vs GitLab Duo: A Comprehensive Comparison of...

As software development accelerates with the increasing demands of continuous...

Nov 15, 2024 · 12

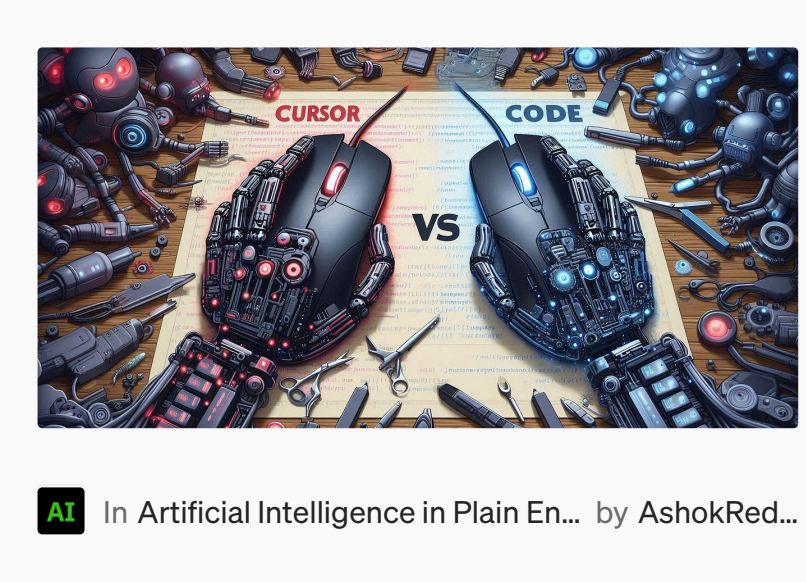


In The AI Spectrum by Gaurav Behere

GitHub Copilot Tricks Every Developer Should Know

Maximizing GitHub Copilot: A Developer's Guide to Chat & Agents

Jan 16 · 60 · 1



In Artificial Intelligence in Plain En... by AshokRed...

Cursor vs. VS Code: The AI-Powered Revolution in Coding

Introduction

Feb 4 · 7

"Tools will come
Tools will go
Only the vibe coder
remains."
-Rick Rubin

Niall McNulty

Vibe Coding

AI-Assisted Coding for Non-Developers

Feb 23 · 10

See more recommendations